CROP SCIENCE · DATA HANDLING IN R · EXERCISE 2

R from zero: Exercise 2, step by step

A beginner's guide to the Tidyverse homework: relational tables, factors and dates, ggplot graphics, long and wide data, and one ANOVA per day.

Who this is for

Someone who has never used R. Every line of code in the exercise is explained.

What you need

The `Ex2_folders.zip` files and either JupyterHub or R + RStudio on your computer.

Based on

The course tutorial by A. Hund & L. Roth (ETH Zurich). All code and output here were run and checked.

Contents

How to use this guide 3

Part 1 What you are actually doing 4

1.1 The experiment in one page 4
1.2 What SPAD measures and why nitrogen matters 5
1.3 How the data are stored: a tiny relational database 5
1.4 Traits, methods and ontologies 7
1.5 Long and wide tables 8
1.6 Just enough statistics: what an ANOVA tells you 8

Part 2 R basics for absolute beginners 9

2.1 R, RStudio, JupyterHub and packages 9
2.2 Where you type, and how you run code 9
2.3 Objects: giving things a name 10
2.4 Functions: the verbs that do the work 11
2.5 Kinds of data: numbers, text, factors, dates 11
2.6 Tables (tibbles) and how to look at them 12
2.7 The pipe |> : "and then" 12
2.8 When something goes wrong 12

Part 3 Setting up 14

3.1 The folders, and what "../" means 14
3.2 Option A: JupyterHub (course server) 14
3.3 Option B: your own computer (R + RStudio) 15
3.4 Three golden rules 15

Part 4 The exercise, step by step 16

Step 1 · Start clean and check where you are 16
Step 2 · Time zone and folder paths 17
Step 3 · Load the tidyverse 17
Step 4 · Read the measurement table 18
Step 5 · Pipes, and a first look at the factor trap 19
Step 6 · Read the other four tables 20
Step 7 · Are the keys really unique? 20
Step 8 · Join the tables into one long table 21
Step 9 · Tell R which columns are categories (factors) 22
Step 10 · Turn text into dates 23
Step 11 · Add a treatment label and save the table 24
Step 12 · Draw the field: your first ggplot 24
Step 13 · SPAD over time, per treatment 27
Step 14 · Q5: your own plot (2 points) 28
Step 15 · From long to wide 30
Step 16 · Do the two days agree? 31
Step 17 · Split the data by day 32
Step 18 · One ANOVA per day, with map() 33
Step 19 · Pull numbers out of the ANOVA tables 34
Step 20 · So what did the experiment show? 35

Part 5 Before you submit 36

5.1 Where each answer goes 36
5.2 Final checks 36

App. A Model answers 37

App. B Error messages and how to fix them 40

App. C Cheat sheet: every function in this exercise 41

App. D Glossary 42

How to use this guide

The guide has five parts. **Read Parts 1 and 2 once** (about 45 minutes, no computer needed). Then open R and work through **Parts 3 and 4 side by side** with your script. Part 5 is a checklist before you hand in. The appendices hold the model answers, an error-message dictionary, a cheat sheet and a glossary.

Coloured boxes mark different kinds of information:

BACKGROUND Why something works the way it does. Optional, but this is where you learn.

YOUR TURN A graded question. Hints are given here; full answers are in Appendix A.

CHECK What you should see if everything worked.

TRAP A place where things silently go wrong. Read these!

TIP A shortcut or a handy habit.

FIX A problem in the exercise files and how to get around it.

Grey code blocks are R code. Dashed blocks show **what R prints back**. Your output may differ in small ways (for example `×` instead of `x`, or slightly different version numbers). That is fine.

TIP **Type the code yourself instead of copying it from the PDF**

Copying from a PDF can turn straight quotes `"` into curly quotes, which R does not understand. Typing also makes you learn much faster. If you copy anyway and get a strange error, retype the quotes.

The grading at a glance

Question	What you do	Points	Type
Q1	Count the comment lines in measurement.csv; say what they are good for	1	text
Q2	Check that the keys of design and trait are unique	1	code + text
Q3	Join the method table to the data	1	code
Q4	Compare the PNG and the SVG file	1	text
Q5	Make your own scatter plot (value vs. fertilizer)	2	code
Q6	Explain what pivot_wider did to the column names	1	text
Q7	Describe what split() produces	1	text
Q8	Collect all F values from the ANOVA tables	1	code
(ungraded)	Look up the SPAD method description	0	text

9 points in total. 5.6 points (60 %) gives a grade 4, 9 points gives a 6.

PART 1 · BACKGROUND

What you are actually doing

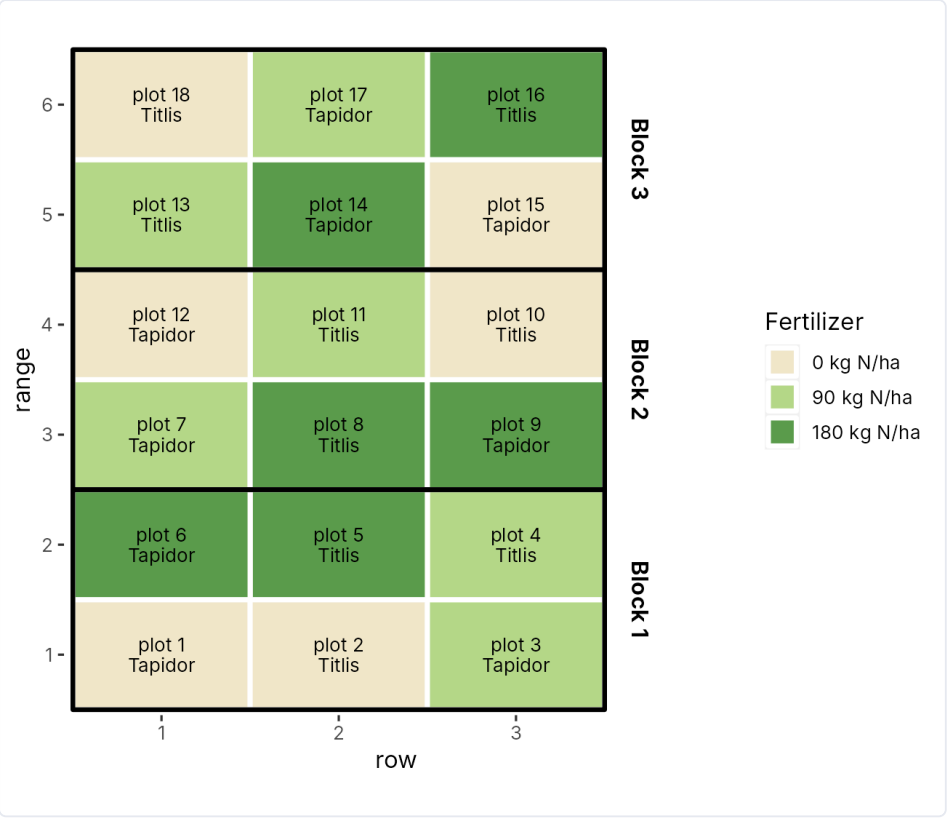
Before touching R, it helps to understand the experiment behind the numbers. Every step of the code makes much more sense once you know what a plot, a block or a SPAD value is.

1.1 The experiment in one page

The data come from a small field trial used on an ETH Zurich plant-nutrition excursion (the data files say April 2014). The trial asks two simple questions:

- **Do two varieties differ** in how green their leaves are? The varieties are called *Tapidor* and *Titlis*.
- **Does nitrogen fertilizer make the leaves greener?** Three rates were applied: 0, 90 and 180 kg of nitrogen per hectare (kg N/ha).

2 varieties × 3 fertilizer rates = **6 treatment combinations**. In the data, each combination is called an **entry** (entry.id 1 to 6). Each entry was grown in **3 blocks**, which gives 6 × 3 = **18 plots**. On two days in a row, 14 and 15 April 2014, the greenness of the leaves of every plot was measured with a SPAD meter. That gives 18 × 2 = **36 measurements**. According to the notes in the raw file, each value is the average of 20 leaves.



The field. 18 plots arranged in 3 rows × 6 ranges. Each thick frame is a block containing every variety × fertilizer combination exactly once. Colours show the fertilizer rate.

Word	Meaning in this experiment
Plot	The smallest piece of field that gets one treatment (one variety, one fertilizer rate). Here: 18 plots, identified by `plot.id`.
Treatment / entry	One combination of variety and fertilizer rate, e.g. "Titlis with 90 kg N/ha". 6 entries, identified by `entry.id`.
Block	A group of neighbouring plots that contains every entry once. 3 blocks = 3 replicates.
Row / range	The x and y coordinates of a plot in the field (like columns and rows in a spreadsheet).
Trait	The property that is measured, here *flag leaf chlorophyll content*.

BACKGROUND **Why blocks? (randomized complete block design)**

A field is never perfectly uniform. The soil may be wetter at one end, or richer where a hedge used to be. If all 180-kg plots stood at the good end, you could not tell whether the leaves were greener because of the fertilizer or because of the soil.

The solution is to divide the field into **blocks** of similar plots and put **every treatment once in every block**, in random order. Differences *between* blocks can then be measured and removed by the statistics. This is what `+ block` does in the ANOVA formula later. The layout is called a *randomized complete block design* (RCBD), and it is the workhorse of field trials.

1.2 What SPAD measures and why nitrogen matters

Chlorophyll is the green pigment that captures light for photosynthesis. Every chlorophyll molecule contains four nitrogen atoms, and most of a leaf's nitrogen sits in the photosynthesis machinery, such as the enzyme Rubisco. A plant that is short of nitrogen therefore makes less chlorophyll, and its leaves turn pale and yellowish. **Leaf greenness is a quick indicator of a plant's nitrogen status.**

A **SPAD meter** (for example the Minolta SPAD-502) is a small hand-held clip that you close over a leaf. It shines two kinds of light through the leaf:

- **red light (650 nm)**, which chlorophyll absorbs strongly, and
- **infrared light (940 nm)**, which chlorophyll does not absorb. It serves as a reference that corrects for leaf thickness.

From the two readings the meter calculates a **unitless index between 0 and 99.9**. Higher means greener. It is not an absolute chlorophyll concentration, but it is fast and does not damage the plant. Breeders use it for phenotyping, and farmers use it to decide whether a field needs more nitrogen. In this data set the values range from 39 to 57.

Expectation before looking at the data: more nitrogen should give greener leaves (higher SPAD), and the varieties might differ. Part 4, Step 20 shows what the data actually say.

1.3 How the data are stored: a tiny relational database

You might expect all the data in one big spreadsheet. Instead they are split into **five small tables** (five CSV files) that point to each other. This is how databases work, and it is called **relational data**.

Table (file)	One row describes…	Rows	Key column
`design.csv`	a plot: which entry grows there, in which block, at which row and range	18	`plot.id`
`entry.csv`	an entry: which variety and which fertilizer rate	6	`entry.id`
`measurement.csv`	one measurement: which plot, which trait, which method, when, and the value	36	`measurement.id`
`trait.csv`	a trait: what is measured (name, short label, ontology link)	2	`trait.id`
`method.csv`	a method: how it is measured (name, label, link, unit)	3	`method.id`

(There is also a sixth file, `SPAD.csv`, the original raw data. The exercise never uses it, so you can ignore it.)

Keys: how the tables find each other

- A **primary key** is a column whose value is **different in every row**, so it identifies exactly one row, like a student ID number. In `entry.csv` that is `entry.id`.
- A **foreign key** is a column that **refers to the primary key of another table**. `design.csv` has a column `entry.id`, which says which entry grows in each plot. It points into `entry.csv`.

design.csv (first 6 of 18 rows)

plot.id	entry.id	block	row	range
1	1	1	1	1
2	2	1	2	1
3	3	1	3	1
4	4	1	3	2
5	6	1	2	2
6	5	1	1	2

entry.csv (all 6 rows)

entry.id	variety	fertilizer
1	Tapidor	0
2	Titlis	0
3	Tapidor	90
4	Titlis	90
5	Tapidor	180
6	Titlis	180

measurement.csv (first 3 of 36 rows)

measurement.id	plot.id	trait.id	method.id	timestamp	value
1	1	1	1	2014.04.14	48.2
2	2	1	1	2014.04.14	49
3	3	1	1	2014.04.14	49.4

trait.csv (selected columns)

trait.id	trait.label	trait.name
1	FI0Chl	Flag leaf chlorophyll content
2	CnpCov	Canopy cover in nadir view

method.csv (selected columns)

method.id	method.label	method.name
1	SPAD001	SPAD measurement on the flag leaf
2	Eye001	Canopy cover rating by eye
3	RGB001	Canopy cover … from RGB images

Reading one measurement through the chain: measurement 1 belongs to **plot 1** → design says plot 1 grows **entry 1** in block 1 → entry 1 is **Tapidor with 0 kg N/ha**. The measurement used **trait 1** (flag leaf

chlorophyll content) and **method 1** (SPAD). Value: 48.2.

BACKGROUND **Why not just one big table?**

- **Each fact is stored once.** "Tapidor" is typed 3 times instead of 36 times, so there is far less room for typos such as "Tapidr".
- **Corrections happen in one place.** If an entry was mislabelled, you fix one row in `entry.csv`, not every measurement.
- **Adding data is easy.** A new measurement day just adds rows to `measurement.csv`.
- **Definitions are reusable.** The trait and method tables can be shared by many experiments.

The price: before analysing, you have to **join** the tables back together. That is exactly what you do in Step 8.

The CSV files also start with a few lines beginning with `#`. These are **comment lines**: notes for humans, not data. You will count them in Q1.

1.4 Traits, methods and ontologies

Plant scientists describe what they measure in a standard way (the **MIAPPE** standard, *Minimum Information About a Plant Phenotyping Experiment*):

Concept	Formula	Example from this exercise
Trait	Entity + Attribute	*Flag leaf* + *chlorophyll content*
Observed variable	Trait + Method + Scale	Flag leaf chlorophyll content, measured with a SPAD meter, as a unitless index

The method matters because the **same trait can be measured in different ways**, and the numbers are not directly comparable. Look at `method.csv`: canopy cover (trait 2) can be rated **by eye** (method 2) or computed **from camera images** (method 3). That is why every value in `measurement.csv` carries both a `trait.id` and a `method.id`.

The **Crop Ontology** (cropontology.org) gives each trait and method a permanent ID and web link, for example `CO_321:0000249` in the Wheat Ontology (CO_321). Labs around the world can then refer to exactly the same definition. Short labels like `Fl0Chl` ("flag leaf chlorophyll") and `SPAD001` are internal abbreviations of the ETH crop science group.

1.5 Long and wide tables

The same numbers can be arranged in two ways:

LONG: one row per measurement (36 rows)

plot.id	variety	date	value
1	Tapidor	2014-04-13	48.2
1	Tapidor	2014-04-14	49.9
2	Titlis	2014-04-13	49.0
2	Titlis	2014-04-14	54.4
…	…	…	…

WIDE: one row per plot (18 rows)

plot.id	variety	value on day 1	value on day 2
1	Tapidor	48.2	49.9
2	Titlis	49.0	54.4
…	…	…	…

- **Long** format is what statistics functions (`aov`) and the plotting package (`ggplot2`) want: every value has its own row, and the other columns say what it is.
- **Wide** format is easier for humans to read and is what you need to **correlate** two measurements (day 1 against day 2).

You build the long table in Step 8 and convert it to wide in Step 15.

1.6 Just enough statistics: what an ANOVA tells you

At the end you run an **analysis of variance (ANOVA)** for each measurement day. It answers: *are the differences between group averages bigger than the random plot-to-plot noise?*

```
aov(value ~ variety * fertilizer + block, data = ...)
```

Read the formula as: "explain `value` by variety, fertilizer, their interaction, and block". `variety * fertilizer` is short for `variety + fertilizer + variety:fertilizer`. The **interaction** `variety:fertilizer` asks: *does the fertilizer effect depend on the variety?*

Column of the ANOVA table	What it means
`Df` (degrees of freedom)	Groups minus 1: variety 1, fertilizer 2, block 2, interaction 1 × 2 = 2; residuals: the rest, 18 − 1 − 7 = 10.
`Sum Sq` , `Mean Sq`	Variation explained by each factor (in total / per Df).
`F value`	Signal ÷ noise: explained variation ÷ leftover random variation. About 1 = just noise; large = probably a real effect.
`Pr(>F)` (p-value)	The probability of getting an F this large if the factor had no effect at all. By convention p < 0.05 is called "significant".

BACKGROUND **Why the code turns numbers into factors**

As a **number**, fertilizer (0, 90, 180) would get a straight-line fit (1 Df). As a **factor** (category), the ANOVA compares three separate group means (2 Df). For blocks the number is just a name: block 3 is not "three times" block 1. So Step 9 turns such columns into factors. This comes back as a trap in Q5.

PART 2 · CRASH COURSE

R basics for absolute beginners

This part teaches you just enough R to understand every line of the exercise. Try the small examples if you already have R open, but reading them is fine too.

2.1 R, RStudio, JupyterHub and packages

Name	What it is
R	A free programming language made for statistics and graphics. It is the "engine" that actually does the work.
RStudio	A program that makes working with R comfortable: an editor for your script, a console, a plot window and a file browser. The "dashboard" around the engine.
JupyterHub	A web server (your course runs one via Moodle) where you can use R in the browser without installing anything.
Package	An add-on that gives R new functions. You **install** it once (`install.packages("name")`) and **load** it at the start of every session (`library(name)`).
tidyverse	A bundle of packages for data work: `readr` (read files), `dplyr` (change tables), `tidyr` (reshape), `ggplot2` (plots), `lubridate` (dates), `purrr` (repeat tasks) and a few more.

2.2 Where you type, and how you run code

In RStudio the window has four panes:

Script editor
Your recipe: the file `Ex2_R_code.R` . You write and edit here, and it is saved. Nothing runs until you tell it to.

Environment
Lists every object (table, value, plot) currently in R's memory.

Console
Where code actually runs and results are printed. The `>` sign means R is waiting for you.

Files · Plots · Help
Browse folders, see your graphics, read help pages.

Action	RStudio	Jupyter notebook
Run the current line (or the whole multi-line command the cursor is in)	**Ctrl + Enter** (Mac: **Cmd + Enter**). The cursor then jumps to the next command.	**Shift + Enter** runs the whole cell
Run several lines	Select them with the mouse, then Ctrl + Enter	Put them in one cell
Start over with a clean R	Session → Restart R	Kernel → Restart

Action	RStudio	Jupyter notebook
Stop / cancel a command	Esc in the console	The ■ stop button

TRAP **The console shows a "+" instead of ">"**

That means R thinks your command is **not finished yet**. Usually a closing bracket `)` or a quote `"` is missing. Click into the console, press **Esc**, fix the line in your script and run it again.

Typing special characters

R code uses a few characters that are awkward on Swiss and German keyboards. RStudio has shortcuts for the two most important ones:

Character	RStudio shortcut	Windows (CH / DE layout)	Mac (CH / DE layout)
`<-` (assign)	Alt + –	type `<` then `-`	Option + –
`|>` (pipe)	Ctrl + Shift + M *	AltGr + 7 (CH) / AltGr + < (DE), then `>`	Option + 7, then `>`
`[ ]`	—	AltGr + ü / AltGr + ¨ (CH); AltGr + 8 / 9 (DE)	Option + 5 / Option + 6
`~` (tilde)	—	AltGr + ^ then Space (CH); AltGr + + (DE)	Option + N, then Space
`` ` `` (backtick)	—	Shift + ^ then Space (CH); Shift + ´ then Space (DE)	Shift + ^ (CH) / Shift + ´ (DE), then Space

* Ctrl + Shift + M (Mac: Cmd + Shift + M) inserts `|>` only if *Tools → Global Options → Code → "Use native pipe operator"* is ticked. Otherwise it inserts `%>%`, which also works. Keyboard layouts differ slightly; if one of these combinations does not work, search the web for "pipe symbol" plus your keyboard.

2.3 Objects: giving things a name

Almost everything in R is stored in an **object** with a name. The arrow `<-` puts something into an object ("assignment"). Read it as "gets".

```
x <- 5          # the object x gets the value 5
x * 2           # use it: R prints 10
x <- 7          # overwrite it: x is now 7
```

OUTPUT

```
[1] 10
```

- The `[1]` in front of the output just means "this is the first value". You can ignore it.
- After `x <- 5`, R prints **nothing**. That is normal: assigning is silent. Type the object's name to see it.
- Names are **case-sensitive**: `Value` and `value` are two different things. Dots are allowed in names, so `dat.long` is one name.

- Everything after `#` on a line is a **comment**. R ignores it, so it is where you write explanations and your text answers.

2.4 Functions: the verbs that do the work

A **function** does something with its input. You call it with its name followed by round brackets. Inside the brackets go the **arguments** (the inputs), separated by commas.

```
mean(c(2, 4, 6))              # c() combines values; mean() averages them -> 4
round(3.14159, digits = 2)    # a named argument -> 3.14
?round                        # opens the help page of round()
```

Arguments can be given **by position** (`round(3.14159, 2)`) or **by name** (`digits = 2`). Names make code easier to read. Note the difference: `=` inside a function call sets an argument, while `<-` creates an object.

2.5 Kinds of data: numbers, text, factors, dates

Type	Looks like	Shown in a table as	Example
Number	`48.2`	`<dbl>` (double) or `<int>` (integer)	SPAD value
Text (character)	`"Tapidor"` (always in quotes!)	`<chr>`	variety names
Factor	looks like text or numbers, but has a fixed set of **levels**	`<fct>`	block 1, 2, 3
Date	`2014-04-14`	`<date>`	measuring day
Date-time	`2014-04-14 00:00:00`	`<dttm>` or `POSIXct`	timestamp
Logical	`TRUE` / `FALSE`	`<lgl>`	result of `5 > 3`

A **vector** is several values of the same type in a row: `c(1, 2, 3)` or `1:10` (the numbers 1 to 10). A table column is a vector.

TRAP **Factors look like numbers, but they are not**

Internally R stores a factor as **level codes** 1, 2, 3, … plus a list of labels. If you turn a factor "back into numbers" with `as.numeric()` , you get the **codes**, not the labels:

```
f <- factor(c(0, 90, 180))
as.numeric(f)                 # 1 2 3      <- the codes, NOT what you wanted
as.numeric(as.character(f))   # 0 90 180   <- correct: factor -> text -> number
```

Remember this two-step trick. You will need it in Q5.

2.6 Tables (tibbles) and how to look at them

Data tables in R are called **data frames**. The tidyverse version is called a **tibble**. Each **row** is one observation, each **column** one variable, and every column has one type.

Command	What it shows
`head(df)`	the first 6 rows
`str(df)`	the **structure**: each column, its type and its first values. Your best friend for checking!
`names(df)`	the column names
`nrow(df)`, `ncol(df)`, `dim(df)`	the number of rows, columns, or both
`df$column`	one column, pulled out as a vector, e.g. `dat.long$value`
`View(df)`	opens the table in a spreadsheet-like viewer (RStudio only)

2.7 The pipe |> : "and then"

The pipe `|>` takes what is on its left and feeds it in as the **first argument** of the function on its right. `x |> f()` is the same as `f(x)`. Read the pipe as **"and then"**:

```
dat.long |>              # take the table dat.long, and then
  filter(DOY == 104) |>  # keep only rows of day-of-year 104, and then
  count(variety)         # count how many rows each variety has
```

Without the pipe you would have to nest the functions inside each other: `count(filter(dat.long, DOY == 104), variety)`. That is harder to read. Two details:

- `==` **compares** ("is equal to?"), while a single `=` sets an argument. Using `=` in `filter()` is a classic beginner error.
- A line ending in `|>` tells R "the command continues on the next line". Never start a new line with `|>`.

2.8 When something goes wrong

Red text in the console is not always an error:

Starts with	Meaning	What to do
`Error` ...	The command **failed**; nothing was done.	Read the message, fix the cause, run the line again.
`Warning` ...	The command ran, but something looks odd.	Read it; often harmless.
anything else	A **message**, i.e. information.	Nothing. For example the start-up text of `library(tidyverse)`.

The two most useful habits:

1. **Read the last line of the error.** It usually names the problem: `object 'dat.long' not found` means you have not created `dat.long` yet, so you skipped a line.
2. **When confused, start fresh:** restart R and run the script from the top, line by line. Most mysterious problems disappear because the objects in memory then match the code again.

Appendix B lists every error you are likely to meet in this exercise, with the fix.

PART 3 · BEFORE YOU START

Setting up

3.1 The folders, and what "../" means

Unzip `Ex2_folders.zip`. You get this structure:

```
Ex2_folders/
├── Scripts/     <- Ex2_R_code.R lives here. R must "stand" in this folder!
├── data/        <- the six input CSV files (never change these)
└── out/         <- empty at first; your results are saved here
__MACOSX/        <- junk created by the Mac zip program: ignore or delete it
```

The script finds files with **relative paths** such as `"../data/"`. The two dots mean **"go up one folder"**. So `../data/measurement.csv` means: from where R currently stands, go up one level, then into `data`, and open `measurement.csv`. This **only works if R's working directory is the `Scripts` folder.**

```
getwd()    # "get working directory": where is R standing right now?
```

CHECK

```
[1] "/home/jovyan/.../Ex2_folders/Scripts"      (your path will look different; it must END
with /Scripts)
```

3.2 Option A: JupyterHub (course server)

1. Open JupyterHub through the link in Moodle and log in.
2. In the file browser on the left, open the Exercise 2 folder and go into `Scripts`. Check that `data` and `out` are next to `Scripts`.
3. Open the exercise file. If it is a **notebook** (`.ipynb`), run each cell with **Shift + Enter**. If it is the `.R` script in the text editor, right-click inside the editor and choose **"Create Console for Editor"**. Then **Shift + Enter** sends the current line to the console.
4. Run `getwd()` and check that it ends with `/Scripts`.

Your course's JupyterHub may look slightly different. The R code itself is identical everywhere; only "how to run a line" changes.

3.3 Option B: your own computer (R + RStudio)

1. **Install R** from cran.r-project.org ("Download R for Windows / macOS").
2. **Install RStudio Desktop** (free) from posit.co/download/rstudio-desktop.
3. Unzip `Ex2_folders.zip` somewhere easy to find, e.g. `Documents/R-Exercise2/`. Delete the `__MACOSX` folder.
4. Start RStudio → *File* → *Open File…* → open `Ex2_folders/Scripts/Ex2_R_code.R`.
5. **Set the working directory:** *Session* → *Set Working Directory* → *To Source File Location.* (A `setwd("…")` line appears in the console; that is fine.)
6. **Install the packages once.** Type these two lines into the **console** (not the script) and wait a few minutes: `install.packages("tidyverse")` and `install.packages("svglite")`.
7. Run `getwd()` and `dir()` to check (see Step 1).

> **TIP** **Never leave install.packages() active in the script**
>
> Installing is needed only once per computer. In the script it stays commented out (`#install.packages("tidyverse")`) so it does not reinstall every time the script runs.

3.4 Three golden rules

1. **Run the script from top to bottom, in order.** Every line builds on the ones before. Skipping a line causes "object not found" errors further down.
2. **When things get weird, restart and rerun from the top.** RStudio: *Session* → *Restart R*, then put the cursor on your current line and press **Ctrl + Alt + B** ("run from beginning to here"). Jupyter: *Kernel* → *Restart & Run All*.
3. **Save often (Ctrl + S) and write text answers as comments** (lines starting with `#`). Then the script still runs from top to bottom without errors, which is important for grading.

PART 4 · WALKTHROUGH

The exercise, step by step

This part follows your script `Ex2_R_code.R` from the first line to the last. For each step you get the code exactly as it appears in the script, a line-by-line explanation, what you should see, and the graded questions in purple boxes.

FIX **Where the answers go: the template has some wrong labels**

The `.R` file has placeholder comments where your answers go, but some are mislabelled. The placeholder **"Add the code for Q 5 here" appears twice**: the second one (right after `head(dat.wide)`) is really for **Q6**. Q7 and Q8 have no question headings at all. Part 5 contains a table that maps every placeholder to its question.

Step 1 · Start clean and check where you are

```
#Clean your workspace
rm(list=ls())
#Find out what's in your working directory
dir()
#setwd("C:/Users/YOUR_PATH") #not needed in JupyterHub on moodle
```

- `ls()` lists all objects in memory; `rm(list = …)` removes them. Together: **empty the memory** so that nothing left over from earlier sessions interferes.
- `dir()` lists the **files in the current folder**.
- The `setwd` line starts with `#`, so it is switched off. You set the folder with the menu instead (Part 3).

CHECK (OTHER FILES MAY BE LISTED TOO)

```
[1] "Ex2_R_code.R"
```

TRAP **If dir() shows "data" "out" "Scripts"**

then R stands one folder too high (in `Ex2_folders`), and every file path will fail. Set the working directory to the `Scripts` folder (Part 3) and run `dir()` again.

Step 2 · Time zone and folder paths

```
#Set the timezone (important for the correct handling of date-time objects)
Sys.setenv(TZ = "Etc/GMT-1")
#define path arguments later used in the script
functionPath <- "../functions/"    # where functions are stored
outputPath <- "../out/"            # where data output and log files can be stored
inputPath <- "../data/"            # input data are in this folder.
```

- `Sys.setenv(TZ = …)` tells R which **time zone** to use when it converts text into dates and times. It matters in Step 10.
- The three `…Path` lines just create **text objects** holding folder names, so the rest of the script can reuse them. `functionPath` is never used in this exercise.

BACKGROUND **"Etc/GMT-1" means UTC plus one hour**

These official time-zone names have a historical quirk: the **sign is reversed**. `Etc/GMT-1` is one hour *ahead* of UTC (world time), i.e. Central European winter time. `Etc/GMT+1` would be one hour behind. A fixed zone like this has no summer-time switch, which is why data loggers are often set to it. You will see a side effect of this in Step 10.

Later the script builds full file names with `paste0()`, which glues pieces of text together without spaces:

```
paste0(inputPath, "measurement.csv")
```

OUTPUT

```
[1] "../data/measurement.csv"
```

Step 3 · Load the tidyverse

```
#install.packages("tidyverse")
library(tidyverse)
```

WHAT YOU SHOULD SEE (VERSION NUMBERS MAY DIFFER)

```
── Attaching core tidyverse packages ──────────────── tidyverse 2.0.0 ──
✔ dplyr     1.1.4     ✔ readr     2.1.5
✔ forcats   1.0.0     ✔ stringr   1.5.1
✔ ggplot2   3.4.4     ✔ tibble    3.2.1
✔ lubridate 1.9.3     ✔ tidyr     1.3.1
✔ purrr     1.0.2
── Conflicts ────────────────────────────────── tidyverse_conflicts() ──
✖ dplyr::filter() masks stats::filter()
✖ dplyr::lag()    masks stats::lag()
```

This is a **message, not an error**. The "Conflicts" part says that the tidyverse brings its own `filter()` function, which hides an older one with the same name. That is exactly what we want.

TRAP **Error: there is no package called "tidyverse"**

The package is not installed on this computer. Run `install.packages("tidyverse")` once in the console, then `library(tidyverse)` again.

Step 4 · Read the measurement table

```
getwd()
measurement <- read_csv(file=paste0(inputPath,"measurement.csv"), comment="#")
head(measurement)
```

- `read_csv()` reads a comma-separated text file and returns a **tibble**.
- `file = …` says which file: `paste0()` builds `"../data/measurement.csv"`.
- `comment = "#"` tells `read_csv` to **ignore everything after a `#`**, so the note lines at the top of the file are skipped.
- The result is stored in a new object called `measurement`, and `head()` shows its first 6 rows.

WHAT YOU SHOULD SEE

```
Rows: 36 Columns: 6
── Column specification ────────────────────────────────────────
Delimiter: ","
chr (1): timestamp
dbl (5): measurement.id, plot.id, trait.id, method.id, value

# A tibble: 6 × 6
  measurement.id plot.id trait.id method.id timestamp  value
           <dbl>   <dbl>    <dbl>     <dbl> <chr>      <dbl>
1              1       1        1         1 2014.04.14  48.2
2              2       2        1         1 2014.04.14  49
3              3       3        1         1 2014.04.14  49.4
4              4       4        1         1 2014.04.14  48.5
5              5       5        1         1 2014.04.14  52.6
6              6       6        1         1 2014.04.14  50.4
```

The "Column specification" text is just information: `read_csv` guessed the type of each column. Notice that `timestamp` is `<chr>`, i.e. **text**. R does not yet know it is a date; that gets fixed in Step 10. The second line of a tibble printout (`<dbl>`, `<chr>`) always shows the column types.

YOUR TURN **Q1 (1 point): How many comment lines does measurement.csv have, and what can they be used for?**

Where: below `#Add answer Q 1 here`. Write the answer as comment lines starting with `#`.

How to see the comment lines: `read_csv` hides them, so look at the raw file. Either open `data/measurement.csv` in a plain **text viewer** (JupyterHub: double-click it in the file browser; RStudio: *Files* pane → click the file → *View File*) rather than Excel, or let R print the first lines of the file as plain text:

```
read_lines(paste0(inputPath, "measurement.csv"), n_max = 5)
```

Count the lines that start with `#` (0.5 p). **Then think** about what they contain and what such information is good for (0.5 p). Imagine you find this file on a hard drive in five years: what would you want to know about it before using the numbers? (Background: section 1.3.)

Step 5 · Pipes, and a first look at the factor trap

```
1:10 |> mean()
mean(1:10)
years <- factor(2008:2012)
as.numeric(as.character(years)) #nesting in the base package
years |> as.character() |> as.numeric() #piping in the tidyverse
```

- The first two lines do the same thing: the mean of 1 to 10, once **with** and once **without** the pipe (see 2.7). Both print `5.5`.
- `factor(2008:2012)` makes a **factor** whose labels look like years.
- The last two lines turn the factor back into real numbers, once nested from the inside out and once with pipes. Both print `2008 2009 2010 2011 2012`.

Try this extra line yourself; it shows *why* the detour via `as.character()` is needed:

```
as.numeric(years)    # without the detour
```

OUTPUT

```
[1] 1 2 3 4 5
```

TRAP **Remember this for Q5**

`as.numeric()` on a factor returns the internal level **codes** (1, 2, 3, ...), not the labels. The correct recipe is **factor** → `as.character()` → `as.numeric()`. The tutorial shows you this example on purpose: you will need it in Q5.

Step 6 · Read the other four tables

```
design <- read_csv(file=paste0(inputPath,"design.csv"), comment="#")
entry <- read_csv(file=paste0(inputPath, "entry.csv"), comment="#")
trait <- read_csv(file=paste0(inputPath, "trait.csv"), comment="#") |>
                  select(trait.id, trait.label, trait.name)
method <- read_csv(file=paste0(inputPath, "method.csv"), comment="#") |>
                  select(method.id, method.label)
```

The first two lines work exactly like Step 4. The trait and method tables are read **and then** (pipe) passed to `select()`, which **keeps only the listed columns**. The links, units and so on are not needed, and fewer columns keep the final table tidy. After this step your Environment contains five tables: `measurement`, `design`, `entry`, `trait`, `method`.

BACKGROUND **Ungraded question: the exact description of the SPAD method**

The method table links to cropontology.org/term/CO_321:0000249. That page (Wheat Ontology, method "**Chl Measurement**", class *Measurement*) says, in short: leaf chlorophyll content is measured quickly and non-destructively with a hand-held optical meter (e.g. the **Minolta SPAD-502**). It measures light transmittance at **650 nm (red)** and **940 nm (infrared)** and corrects for leaf thickness. Readings are a relative **chlorophyll concentration index (0–99.9)**, not absolute chlorophyll values, and are typically taken on the fully expanded **flag leaf**. Reference: Pask et al. 2012, chapter 9.

Step 7 · Are the keys really unique?

A join (next step) only works properly if each key value appears **once** in the table you look things up in. Otherwise rows get duplicated. So first we check.

```
entry |>
  count(variety) |>
  filter(n > 1)
```

Read it as: *take `entry`, and then count how often each variety occurs, and then keep only the rows where that count `n` is bigger than 1.* `count()` produces a small table with one row per value and a column `n`. `filter()` keeps rows that meet a condition. **Any row that survives is a duplicate.**

WHAT YOU SHOULD SEE

```
# A tibble: 2 × 2
  variety     n
  <chr>   <int>
1 Tapidor     3
2 Titlis      3
```

Each variety appears 3 times, so `variety` is **not** a unique key. That is fine, because it was never meant to be one. Now the real key:

```
entry |>
  count(entry.id) |>
  filter(n > 1)
```

WHAT YOU SHOULD SEE

```
# A tibble: 0 × 2
# ℹ 2 variables: entry.id <dbl>, n <int>
```

Zero rows = no duplicates = `entry.id` is unique. That is the result you hope for when checking a primary key.

YOUR TURN **Q2 (1 point): Check the unique keys of the design and trait tables**

Where: below `#Add answer Q 2 here .`

What to do: copy the pattern above (the second block, with `count` and `filter`) twice and change two things each time: the **table** (`design`, then `trait`) and the **key column**. Which column is the primary key of `design`? (Hint: section 1.3. It is *not* `entry.id`.) Which one for `trait`?

What to write: the code, plus a comment line after each block stating whether the key is unique. If the output is `# A tibble: 0 × 2`, it is unique.

TIP **Understanding check (not required)**

Try `design |> count(entry.id)`. Every entry appears 3 times. That is correct, because in `design` the column `entry.id` is a *foreign* key: each entry is grown in 3 blocks.

Step 8 · Join the tables into one long table

A **left join** works like a lookup: *take every row of the left table, find the matching row in the right table via the key, and glue the right table's columns on.* Rows of the left table are never lost. If there is no match, the new columns are filled with `NA` (missing).

```r
#Do a left join
dat.long <- design |>
  left_join(entry, by="entry.id") |>
  left_join(measurement, by="plot.id") |>
  left_join(trait, by="trait.id")
```

After...	Rows × columns	What happened
`design`	18 × 5	one row per plot
`+ entry` (by entry.id)	18 × 7	each plot gets its variety and fertilizer
`+ measurement` (by plot.id)	**36** × 12	each plot has **2** measurements (2 days), so every plot row appears twice
`+ trait` (by trait.id)	36 × 14	each measurement gets the trait label and name
`+ method` (by method.id)	36 × 15	← this is your Q3

`by = "entry.id"` names the **key column** that both tables share. Notice how the information "plot 1 is Tapidor, 0 kg N, block 1" is now **repeated** in two rows. That redundancy is the price of the long format, and it is what makes plotting and statistics easy.

YOUR TURN **Q3 (1 point): Add the method table to dat.long**

Where: directly below `#Add answer Q 3 here`, i.e. right after the join above. **Not later!** (See the trap below.)

Ingredients: the table to add is `method`, and the key column it shares with `dat.long` is `method.id` (look at `names(method)`). Take the existing `dat.long`, pipe it into one more `left_join(…)`, and store the result as `dat.long` again (overwrite it with `<-`).

Check: `names(dat.long)` now ends with `"method.label"`, and `dim(dat.long)` gives `36 15`.

TRAP **The position of Q3 matters**

The very next step turns all `.id` columns into factors. If you join `method` **after** that step, `dat.long$method.id` is a factor but `method$method.id` is still a number, and R refuses:

```
Error: Can't join x$method.id with y$method.id due to incompatible types.
```

Fix: put the join directly under the Q3 placeholder, restart R and run from the top again.

Step 9 · Tell R which columns are categories (factors)

```r
#Identify all ids using the grep function.
str(dat.long)
ids<-names(dat.long)[grep(".id", names(dat.long))]

#Add additional columns which are considered factors
Factors <- c(ids, "block", "row", "range", "variety", "fertilizer")

dat.long<-dat.long |> dplyr::mutate(across(all_of(Factors), as.factor))
str(dat.long)
```

Line by line:

- `str(dat.long)` shows the structure **before** the change: `block`, `row`, `range`, `fertilizer` and all the IDs are `num` (numbers).
- `grep(".id", names(dat.long))` searches the column names for the pattern ".id" and returns the **positions** of the matches. `names(dat.long)[…]` then picks out those names. Result: `ids` holds `"plot.id" "entry.id" "measurement.id" "trait.id" "method.id"`.
- `c(ids, "block", …)` combines these into one list of column names called `Factors`.
- `mutate()` changes columns; `across(all_of(Factors), as.factor)` means "for **all** the columns named in `Factors`, apply `as.factor`". `dplyr::` explicitly says "the `mutate` from the dplyr package". It is not strictly needed here.

CHECK (SECOND STR: AFTER THE CHANGE)

```
tibble [36 × 15] (S3: tbl_df/tbl/data.frame)
 $ plot.id       : Factor w/ 18 levels "1","2","3","4",..: 1 1 2 2 3 3 4 4 5 5 ...
 $ entry.id      : Factor w/ 6 levels "1","2","3","4",..: 1 1 2 2 3 3 4 4 6 6 ...
 $ block         : Factor w/ 3 levels "1","2","3": 1 1 1 1 1 1 1 1 1 1 ...
 $ row           : Factor w/ 3 levels "1","2","3": 1 1 2 2 3 3 3 3 2 2 ...
 $ range         : Factor w/ 6 levels "1","2","3","4",..: 1 1 1 1 1 1 2 2 2 2 ...
 $ variety       : Factor w/ 2 levels "Tapidor","Titlis": 1 1 2 2 1 1 2 2 2 2 ...
 $ fertilizer    : Factor w/ 3 levels "0","90","180": 1 1 1 1 2 2 2 2 3 3 ...
 $ measurement.id: Factor w/ 36 levels "1","2","3","4",..: 1 19 2 20 3 21 4 22 5 23 ...
 $ trait.id      : Factor w/ 1 level "1": 1 1 1 1 1 1 1 1 1 1 ...
 $ method.id     : Factor w/ 1 level "1": 1 1 1 1 1 1 1 1 1 1 ...
 $ timestamp     : chr [1:36] "2014.04.14" "2014.04.15" "2014.04.14" "2014.04.15" ...
 $ value         : num [1:36] 48.2 49.9 49 54.4 49.4 50.4 48.5 54.1 52.6 53.2 ...
 $ trait.label   : chr [1:36] "Fl0Chl" "Fl0Chl" "Fl0Chl" "Fl0Chl" ...
 $ trait.name    : chr [1:36] "Flag leaf chlorophyll content" ...
 $ method.label  : chr [1:36] "SPAD001" "SPAD001" "SPAD001" "SPAD001" ...
```

Note the codes after the colon: `fertilizer` shows `1 1 1 1 2 2 …`. Those are the level codes from section 2.5, not the kilograms. `method.id` and `method.label` only appear if you did Q3 correctly.

BACKGROUND **A small flaw in grep(".id", ...), for the curious**

`grep` treats its pattern as a *regular expression*, where a dot means "**any character**". So ".id" would also match a column called "val**id**" or "Lip**id**". It works here only because no such column exists. Safer options are `grep(".id", names(dat.long), fixed = TRUE)` or `ends_with(".id")` . You do **not** need to change this for the exercise.

Step 10 · Turn text into dates

```
dat.long$timestamp <- as.POSIXct(strptime(dat.long$timestamp, "%Y.%m.%d"))
dat.long$date <-as.Date(dat.long$timestamp)

#Lubridate offers useful functions to extract components from dates:
  #The week of the year
  week(dat.long$timestamp)
  #The quarter of the year
  lubridate::quarter(dat.long$timestamp)
  #We add a new column "DOY" to the tibble
  dat.long$DOY <- lubridate::yday(dat.long$timestamp) #The day of the year
  str(dat.long$DOY)
```

- `strptime(text, format)` reads text as a date-time. The **format** describes how the text is built: `%Y` = 4-digit year, `%m` = month, `%d` = day, and the dots are the literal separators used in the file (`2014.04.14`). (The tutorial text says "-" is the separator; the file actually uses dots.)
- `as.POSIXct()` stores it as a date-time (internally: seconds since 1 Jan 1970).
- `dat.long$timestamp <- …` **overwrites** the column. `dat.long$date <- …` **creates** a new column holding just the date.
- `week()` , `quarter()` and `yday()` from the lubridate package extract the week of the year (15), the quarter (2) and the **day of year** (DOY: 14 April = 31 + 28 + 31 + 14 = day 104).

CHECK

```
 num [1:36] 104 105 104 105 104 105 104 105 104 105 ...
```

TRAP **The date column is one day early (and that is expected)**

Run `head(dat.long$date)`. The file says **14** and **15** April, but `date` shows **2014-04-13** and **2014-04-14**, while `DOY` correctly says 104 and 105. Why?

1. `strptime` reads "2014.04.14" as **midnight, 14 April**, in the time zone from Step 2 (UTC+1).
2. Midnight in UTC+1 is the same moment as **23:00 on 13 April in UTC**.
3. `as.Date()` converts using **UTC** by default, and so lands on 13 April.

What to do: leave the code as it is. The tutorial's expected outputs (the column names in Step 15 and the panel labels in Q5) all show 2014-04-13 and 2014-04-14, so your results will match. Just know why. The "proper" fix would be `as.Date(dat.long$timestamp, tz = "Etc/GMT-1")`. Time zones are a classic source of off-by-one errors with logger data.

Step 11 · Add a treatment label and save the table

```
dat.long <- dat.long |> mutate(treat = paste(fertilizer, variety, sep="."))
write_rds(dat.long, file=paste0(outputPath, "dat.long.rds"))
```

- `paste(a, b, sep = ".")` glues values with a dot in between: `"0.Tapidor"`, `"90.Titlis"`, ...
  `mutate(treat = …)` stores this as a new column `treat`, which gives nice labels for the plots.
- `write_rds()` saves the whole table into `out/dat.long.rds`. The `.rds` format is R's own file type. Unlike CSV, it **remembers the column types** (factors, dates), so you can load the table later exactly as it is now.

FIX **The file "dat.longFull.rds" is missing: add one line here**

In Step 15 the script loads `../out/dat.longFull.rds`. That file is **not in your zip** (the `out` folder is empty), so the script stops with:

```
Error in readRDS(con, refhook = refhook) : cannot open the connection
```

```
cannot open file '../out/dat.longFull.rds': No such file or directory
```

"Full" means "with the method table joined". After your Q3, your own `dat.long` **is** that full version. So add this line directly **below** the `write_rds` line above:

```
write_rds(dat.long, file=paste0(outputPath, "dat.longFull.rds"))  # saves my full table (incl. method) under the name used later
```

(If your JupyterHub `out` folder already contains this file from the teachers, the extra line simply replaces it with your equivalent version. Either way it works.)

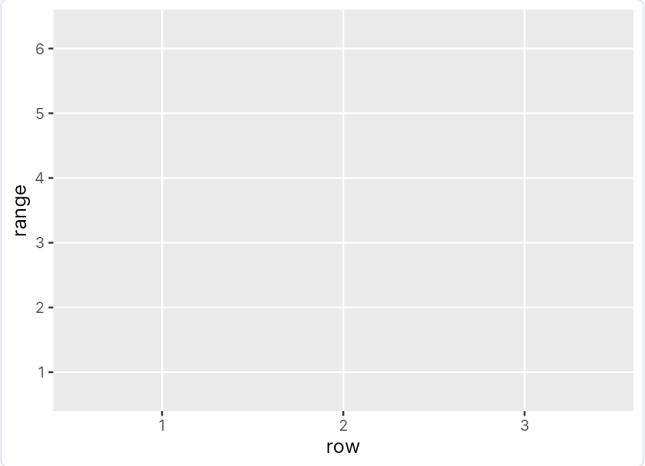
Step 12 · Draw the field: your first ggplot

ggplot2 builds a graphic in **layers** that you add with `+`:

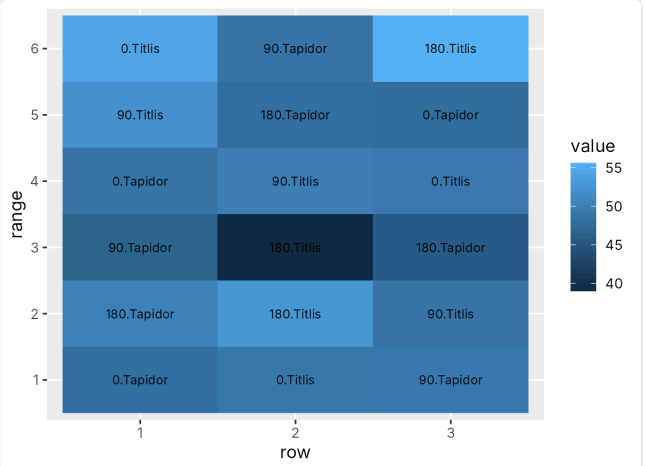
Layer	What it defines	Here
`ggplot(data, aes(…))`	the data and the **aesthetic mapping**: which column goes to which visual property (x, y, colour, fill, label…)	x = row, y = range
`geom_…()`	the **geometry**: what is drawn (points, lines, tiles, text)	`geom_raster` (coloured tiles), `geom_text`
`facet_grid()`	splits the plot into **panels** by a variable	one panel per day
`theme()`, `scale_…()`	the look: fonts, angles, colours	later

```
#Only the day of the year (DOY) 104
    p1 <- ggplot(dat.long |> filter(DOY == 104), aes(x=row, y=range))
    p2 <- p1 + geom_raster(aes(fill=`value`))
    p3 <- p2 + geom_text(aes(label=treat), size=3)
plot(p1)
plot(p2)
plot(p3)
```

- `dat.long |> filter(DOY == 104)` uses only the 18 rows of the first day.
- `p1` is only the empty **canvas**: axes, but nothing drawn.
- `p2` adds coloured tiles whose **fill** colour shows the SPAD value. (The backticks around `value` are not needed, but they do no harm.)
- `p3` writes the treatment label in each tile; `size = 3` sets the text size.
- Making a plot object does not show it. `plot(p1)` (or just typing `p1`) displays it.



`plot(p1)` : the empty canvas



`plot(p3)` : tiles + labels

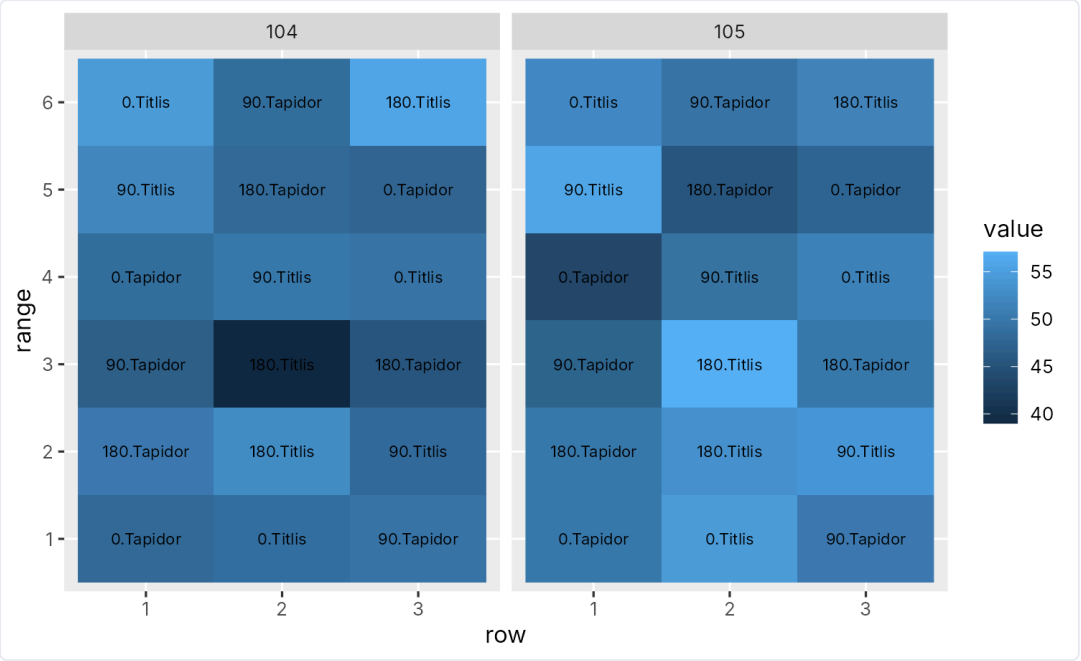
Your first plots. The darkest tile (row 2, range 3) is plot 8 with a SPAD of only 39.0 on day 1. Keep it in mind.

TIP **Your plots look a little different from the tutorial's pictures, and that is fine**

In the tutorial images the axes show 1.0, 1.5, 2.0, ... Those pictures were evidently made while `row` and `range` were still numbers. Yours were turned into factors in Step 9, so the axes show the categories 1, 2, 3 (and 1 to 6). Nothing is wrong.

```r
#Both days together - this make only sense in combination with facet_grid()
    p_facet <- ggplot(dat.long , aes(x=row, y=range)) +
          geom_raster(aes(fill=`value`)) +
          geom_text(aes(label=treat), size=3)
    p_facet <- p_facet + facet_grid( ~ DOY)
    plot(p_facet)
```

`facet_grid( ~ DOY)` makes **one panel per value of DOY**, side by side. (Anything left of `~` would become rows of panels, and anything right of it becomes columns.)



`plot(p_facet)`: the field on both days. Plot 8 jumps from the darkest tile on day 104 to the brightest on day 105. A real plant cannot change that much overnight, so this is most likely a measurement error.

```r
p<-ggplot(dat.long, aes(row, range)) +
  geom_raster(aes(fill = `value`)) +
  geom_text(aes(label=treat), size=3) +
  facet_grid( ~ DOY)

  ggsave(p, filename = paste0(outputPath, "rastergraph.png"))
  ggsave(p, filename = paste0(outputPath, "rastergraph.svg"), device="svg")
```

The same plot in a single command, saved twice with `ggsave()`: once as **PNG** and once as **SVG**, both into the `out` folder. The message `Saving 7 x 7 in image` is normal. Your numbers may differ, because `ggsave` uses the size of your current plot window.

TRAP **Error: there is no package called "svglite"**

Saving as SVG needs the extra package **svglite**. Run `install.packages("svglite")` once, then run the `ggsave` line again.

YOUR TURN **Q4 (1 point): Compare the PNG and the SVG file. Which one is better for further processing or scaling?**

Where: below `#Add answer Q 4 here`, as comments.

What to do: open both files from the `out` folder (JupyterHub: double-click in the file browser, or right-click the image → "Open Image in New Tab"; own computer: open them in a web browser). **Zoom in a lot** (Ctrl + mouse wheel) on some text. Also open the `.svg` once in a **text editor**: what do you see inside? Compare the file sizes as well.

Then read the background box below, describe the differences in your own words, and say which format you would choose for further editing or scaling, and **why**.

BACKGROUND **Raster vs. vector graphics**

A **raster** image (PNG, JPG) is a grid of coloured **pixels**. A PNG saved at 7 × 7 inches and 300 dots per inch has 2100 × 2100 pixels, and no more. Enlarge it beyond that and you see the individual pixels: edges get blocky or blurry.

A **vector** image (SVG, PDF, EPS) stores **instructions**: "draw a rectangle here, write the text 0.Tapidor there in font size 8.5". The viewer redraws it at any size, so it stays sharp, and text remains real, editable text. SVG is plain text (XML) inside, which is why you can open it in a text editor.



PNG, zoomed in strongly



SVG, same zoom

The same piece of `rastergraph`, enlarged: the raster image shows its pixels, while the vector image is redrawn sharply.

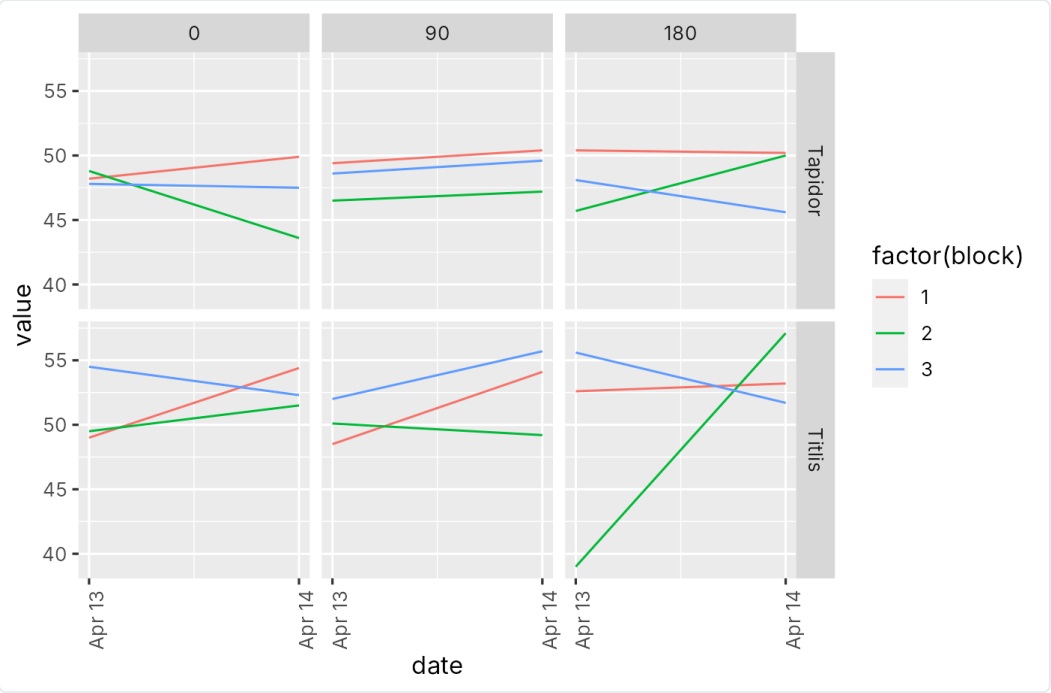
Step 13 · SPAD over time, per treatment

```
#Prepare the panel
p <- ggplot(dat.long, aes(x=date, y=value)) #Define what you want to plot
#plot the lines and define groups to be coloured
p <- p + geom_line(aes(x=date, y=value, group=plot.id, colour=factor(block)))
#Set the defined colours
p <- p + scale_color_discrete()
#Define a different panel for each Variety-by-fertilizer treatment
p <- p + facet_grid(variety ~ fertilizer)
#Rotate the x-axis to be readable.
p<-p + theme(axis.text.x = element_text(angle = 90))
```

- `geom_line` draws lines. `group = plot.id` says **which points belong together**: the two days of the same plot are connected by one line.
- `colour = factor(block)` colours each line by its block. (`block` is already a factor; `factor()` does no harm.)
- `scale_color_discrete()` uses the default colour palette for categories.
- `facet_grid(variety ~ fertilizer)`: **rows** of panels = variety, **columns** = fertilizer, giving 2 × 3 panels.
- `theme(axis.text.x = element_text(angle = 90))` turns the x-axis labels by 90° so they do not overlap.

TRAP **The script never shows this plot**

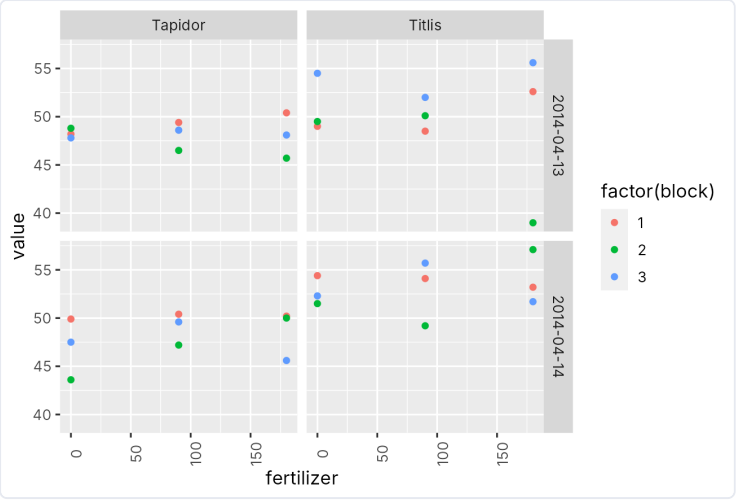
The template builds `p` but has no `plot(p)` line. Add `plot(p)` (or just `p`) at the end if you want to see it.



Each line is one plot, from day 1 to day 2. The steep green line (Titlis, 180 kg N, block 2) is plot 8 again.

Step 14 · Q5: your own plot (2 points)

The task: *"Look at value (y) vs. fertilizer (x), grouped by date and variety. Use `geom_point` instead of `geom_line`."* Your result should look like the target figure. Build it in small steps and look at the plot after each one.



The target figure (as in the tutorial). Note: points; x = fertilizer with a *numeric* axis (0, 50, 100, 150); colours = block; panel rows = date, panel columns = variety; x labels rotated.

a) Start with the canvas and the points

Following the pattern of Step 13: `ggplot(dat.long, aes(x = …, y = …))` with the right columns, plus `geom_point()`. Look at the result.

b) Colour by block

Inside `geom_point(aes(…))`, add the same colour mapping that the line plot uses.

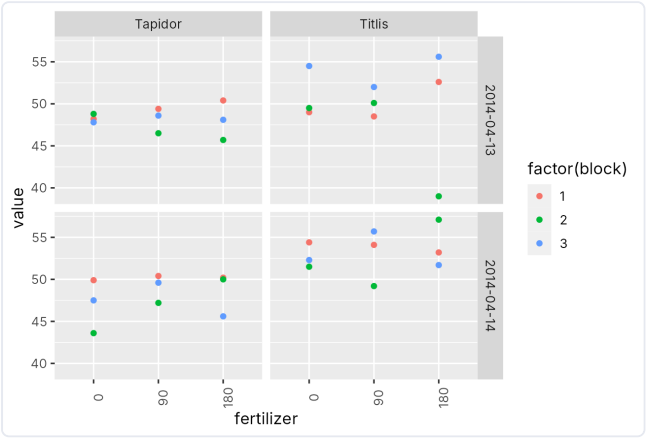
c) Panels: rows = date, columns = variety

Look at the target: the date labels are on the **right** (so date defines the **rows**), and the variety labels are on **top** (the **columns**). The `facet_grid()` formula is `rows ~ columns`. Compare with Step 13, where it was `variety ~ fertilizer`.

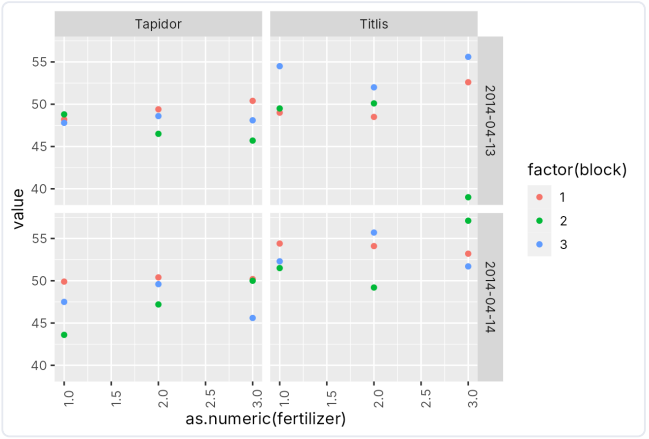
d) Rotate the x labels

Copy the `theme(…)` line from Step 13.

e) Compare with the target and spot the difference



What you get so far: 0, 90, 180 are evenly spaced categories



The trap: `as.numeric(fertilizer)` gives 1, 2, 3

Two plots that are almost, but not quite, right.

Your x-axis shows 0, 90, 180 as **evenly spaced categories**, because Step 9 turned `fertilizer` into a factor. The target has a real **number axis** (90 sits nearer to 0 than 180 does, with ticks at 0, 50, 100, 150). So for this plot, fertilizer must become a number again, **correctly**: see Step 5 and section 2.5. `as.numeric(fertilizer)` alone gives the level codes 1, 2, 3 (right-hand picture above).

f) Convert inside a pipe, without changing dat.long

Change the column **only for this plot**: start from `dat.long`, pipe it into `mutate(fertilizer = …)` to replace the column with its numeric version, and pipe the result straight into `ggplot()`. Keeping the name `fertilizer` also gives the axis the right title. The shape of the code:

```
p <- dat.long |>
  mutate(fertilizer = ...) |>          # your conversion goes here (see Step 5)
  ggplot(aes(x = ..., y = ...)) +
  ...                                   # the layers from a) to d), joined with +
plot(p)
```

TRAP **Three things that go wrong in Q5**

1. **Do not overwrite** `dat.long` with the numeric fertilizer (`dat.long <- dat.long |> mutate(…)`). The ANOVA in Step 18 needs fertilizer as a factor. If it becomes a number, the ANOVA silently changes (1 instead of 2 degrees of freedom).
2. `|>` **versus** `+` **:** the pipe `|>` passes **data** into a function; `+` adds **layers** to a ggplot. Before `ggplot()` use `|>`; after it use `+`. Mixing them up gives errors like " `` `mapping` must be created by `aes()` ". Also never start a line with `+`.
3. **Call it** `p` **and show it.** The tutorial's next chunk is `plot(p)`, but your template does not contain that line, so add `plot(p)` yourself.

YOUR TURN **Q5 checklist (2 points)**

- ☐ points, not lines
- ☐ x = fertilizer as **numbers** (axis 0–180), y = value
- ☐ colour = block, legend titled `factor(block)`
- ☐ panel rows = date (2014-04-13 and 2014-04-14), panel columns = Tapidor and Titlis
- ☐ rotated x labels
- ☐ stored in `p` and shown with `plot(p)`, placed below the **first** "Add the code for Q 5 here"

Step 15 · From long to wide

```
dat.long<-read_rds(file=paste0(outputPath, "dat.longFull.rds"))

dat.wide<-dat.long |> select(-measurement.id, -timestamp , -DOY)

dat.wide<-dat.wide |> pivot_wider(names_from = c(trait.label, method.label, date),
                                  values_from=value)
head(dat.wide)
```

- `read_rds()` loads the saved full table. This is where the missing-file problem would hit you; see the fix in Step 11.

- `select(-measurement.id, -timestamp, -DOY)`: a **minus** in `select` means "**drop** this column".
- `pivot_wider()` turns the long table into a wide one. `names_from` says which column(s) supply the **names of the new columns**. `values_from` says which column supplies the **numbers that fill them**.

BACKGROUND **Why the three columns must be dropped first**

`pivot_wider` merges rows that are identical in **all other columns** into one. The two measurements of plot 1 differ in `value` and `date` (that is the point), but they also differ in `measurement.id`, `timestamp` and `DOY`. If those stay, R cannot merge the rows. You get 36 rows instead of 18, each half-empty (`NA`). Try it once without the `select` line and look!

CHECK (THE COLUMN ORDER MAY DIFFER FROM THE TUTORIAL; THAT IS FINE)

```
# A tibble: 6 × 13
  plot.id entry.id block row   range variety fertilizer trait.id method.id
  <fct>   <fct>    <fct> <fct> <fct> <fct>   <fct>      <fct>    <fct>
1 1       1        1     1     1     Tapidor 0          1        1
2 2       2        1     2     1     Titlis  0          1        1
3 3       3        1     3     1     Tapidor 90         1        1
4 4       4        1     3     2     Titlis  90         1        1
5 5       6        1     2     2     Titlis  180        1        1
6 6       5        1     1     2     Tapidor 180        1        1
# ℹ 4 more variables: trait.name <chr>, treat <chr>,
#   `Fl0Chl_SPAD001_2014-04-13` <dbl>, `Fl0Chl_SPAD001_2014-04-14` <dbl>
```

YOUR TURN **Q6 (1 point): What happened to the content of trait.label, method.label and date?**

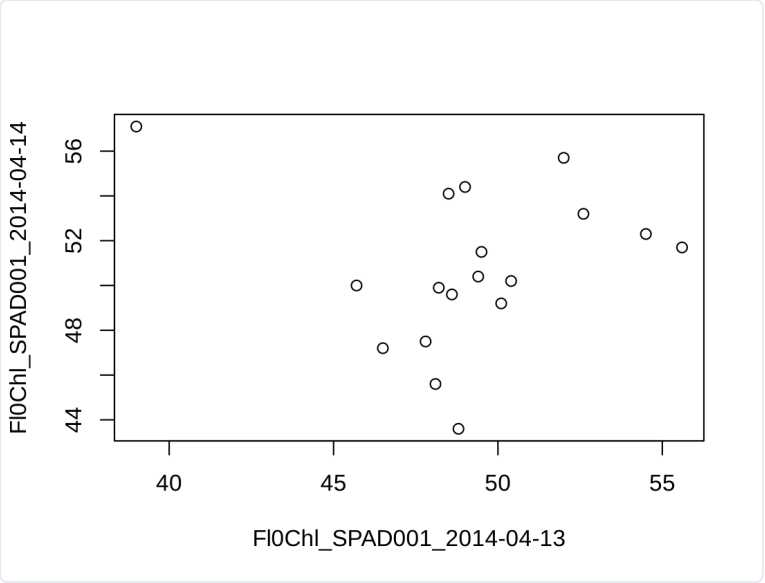
Where: below the **second** `#Add the code for Q 5 here` (right after `head(dat.wide)`); this placeholder is really Q6. Write the answer as comments.

Investigate: run `names(dat.long)` and `names(dat.wide)` and compare them. Look closely at the last two column names of `dat.wide`. Where do the parts of the name `Fl0Chl_SPAD001_2014-04-13` come from? What character separates them? Do the columns `trait.label`, `method.label`, `date` and `value` still exist? Where did the SPAD numbers go? How many rows does `dat.wide` have compared with `dat.long`, and why?

Step 16 · Do the two days agree?

```
TraitsWide<-dat.wide |> select(contains("Fl0Chl"))
plot(TraitsWide)
```

`select(contains("Fl0Chl"))` keeps only the columns whose **name contains** "Fl0Chl", which here means the two SPAD columns. `plot()` on a table with two columns draws a **scatter plot**: one point per plot, day 1 on x, day 2 on y. (This is base R's `plot`, not ggplot.)



SPAD on day 1 (x) vs. day 2 (y). Each point is one plot. The lonely point at the top left is plot 8.

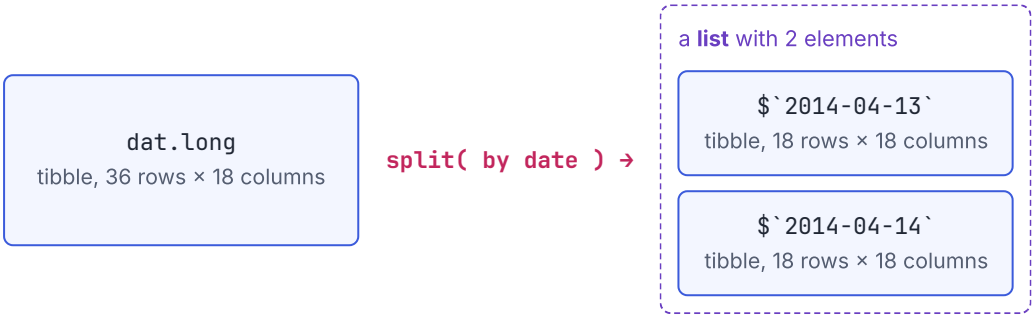
BACKGROUND **What this tells you**

If the measurements were reliable and the plots really differed, the points would line up along a rising diagonal: greener plots on day 1 would also be greener on day 2. They do not. The correlation is about zero (`cor(TraitsWide)` gives r ≈ −0.03). One reason is plot 8 (39.0 → 57.1), very likely a measurement error. This is why you **look at the data before running statistics**.

Step 17 · Split the data by day

Next we want one ANOVA **per day**. The first step is to cut the table into one piece per date:

```
test<-dat.long |> split(dat.long$date)
str(test)
```



WHAT YOU SHOULD SEE (SHORTENED)

```
List of 2
 $ 2014-04-13: tibble [18 × 18] (S3: tbl_df/tbl/data.frame)
  ..$ plot.id       : Factor w/ 18 levels "1","2","3","4",..: 1 2 3 4 5 6 7 8 9 10 ...
  ..$ entry.id      : Factor w/ 6 levels "1","2","3","4",..: 1 2 3 4 6 5 3 6 5 2 ...
  ..                  (… 16 more columns …)
 $ 2014-04-14: tibble [18 × 18] (S3: tbl_df/tbl/data.frame)
  ..$ plot.id       : Factor w/ 18 levels "1","2","3","4",..: 1 2 3 4 5 6 7 8 9 10 ...
  ..                  (…)
```

BACKGROUND **What is a list?**

A **list** is R's most flexible container: an ordered collection of objects of **any** kind (tables, numbers, plots, even other lists). Elements can have names. You get one element out with double square brackets, by position `test[[1]]` or by name `test[["2014-04-13"]]`.

YOUR TURN **Q7 (1 point): Describe what you get when you split the tibble by date**

Where: below `#Type your answer here.`, as comments.

Investigate: run `class(test)`, `length(test)`, `names(test)`, `str(test, max.level = 1)` and `test[[1]]`. Then describe: What kind of object is `test`? How many elements does it have, and why that number? What are the elements called? What is each element, and how many rows and columns does it have? Do the pieces together still contain all 36 rows?

TIP **A detail for the curious**

Inside the pipe, `split(dat.long$date)` grabs the date column from the object `dat.long` outside the pipe, not from what flows through the pipe. Here both are the same table, so it works. `split(~ date)` would be the cleaner version.

Step 18 · One ANOVA per day, with map()

```
aov1<-dat.long |> split(dat.long$date) |>
  purrr::map(function(df) aov(value ~ variety * fertilizer + block, data=df))
#You can use "\" as anonyoumous function shortcut.and the dot "." for the dataframe.
aov1<-dat.long |> split(dat.long$date) |>
  map( \(.) aov(value ~ variety * fertilizer + block, data=.))

#this is how the "%>%" operator would be used:
#aov1<-dat.long %>% split(dat.long$date) %>%
#map(~aov(value ~ variety * fertilizer + block, data=.)) #

aov1
```

- `map(list, f)` **applies the function** `f` **to every element of the list** and returns a new list with the results. Here: one ANOVA for each day. It replaces a "for loop".
- `function(df) aov(…, data = df)` is an **anonymous function**: a little function without a name. `df` is a placeholder for "the current piece", i.e. the table of one day.
- `\(.) aov(…, data = .)` is exactly the same thing written shorter: `\(x)` is short for `function(x)`, and here the placeholder happens to be called `.`.
- The three lines starting with `#` show the older `%>%` style. They are switched off; ignore them.
- Both versions create the same `aov1`; the second simply overwrites the first.

WHAT YOU SHOULD SEE

```
$`2014-04-13`
Call:
   aov(formula = value ~ variety * fertilizer + block, data = .)

Terms:
                  variety fertilizer     block variety:fertilizer Residuals
Sum of Squares   16.62722    3.44111  63.52778            2.28111 133.65889
Deg. of Freedom         1          2         2                  2        10

Residual standard error: 3.655939
Estimated effects may be unbalanced

$`2014-04-14`
   (… same layout, Sum of Squares 113.50222 6.97333 16.41333 2.75111 65.84000 …)
```

Check: the degrees of freedom are **1, 2, 2, 2, 10**, exactly as calculated in section 1.6. If you see a 1 for fertilizer, then fertilizer is a number instead of a factor (see the Q5 trap). "Estimated effects may be unbalanced" is a standard note you can ignore.

Step 19 · Pull numbers out of the ANOVA tables

```r
aov.tab<-aov1 |> map(.f=~summary(.x))
aov.tab<-aov1 |> map(summary) #same

#Access the ANOVA table to do some summary statistics
str(aov.tab) #A list in a list
str(aov.tab[[1]][[1]]$`Pr(>F)`) #
```

`summary()` turns each ANOVA into the familiar table. Printing `aov.tab` shows:

PRINT(AOV.TAB)

```
$`2014-04-13`
                   Df Sum Sq Mean Sq F value Pr(>F)
variety             1  16.63   16.63   1.244  0.291
fertilizer          2   3.44    1.72   0.129  0.881
block               2  63.53   31.76   2.376  0.143
variety:fertilizer  2   2.28    1.14   0.085  0.919
Residuals          10 133.66   13.37

$`2014-04-14`
                   Df Sum Sq Mean Sq F value  Pr(>F)
variety             1 113.50  113.50  17.239 0.00197 **
fertilizer          2   6.97    3.49   0.530 0.60450
block               2  16.41    8.21   1.246 0.32861
variety:fertilizer  2   2.75    1.38   0.209 0.81491
Residuals          10  65.84    6.58
```

To reach one column you need to go down **three levels**. Read `aov.tab[[1]][[1]]$`Pr(>F)`` from left to right:

Piece	Gives you
`aov.tab`	the list of 2 days

Piece	Gives you
`[[1]]`	the summary of the **first day**, itself a list with 1 element (hence "a list in a list")
`[[1]]`	that one element: the **ANOVA table** (a data frame)
`` $`Pr(>F)` ``	the column of p-values. The **backticks** are needed because the name contains special characters: brackets, `>`, and in other columns spaces.

```
Pr.val<-aov.tab |> map(function(x){x[[1]]$`Pr(>F)`})
Pr.val
```

WHAT YOU SHOULD SEE

```
$`2014-04-13`
[1] 0.2907863 0.8806472 0.1430874 0.9188675        NA

$`2014-04-14`
[1] 0.001973723 0.604497550 0.328610606 0.814910513          NA
```

`map` again: for each day `x`, take the table `x[[1]]` and pull out the `Pr(>F)` column. The `NA` at the end belongs to the *Residuals* row, which has no test. That is normal.

YOUR TURN **Q8 (1 point): Collect all the F values from aov.tab**

Where: below `#Type your code here` (the last line of the script).

How: this is the `Pr.val` line with **two small changes**: a new object name (e.g. `F.val`) and a different column. What is the exact name of the F column in the printed table? It contains a **space**, so it needs backticks too. Print your result afterwards.

Check: your numbers should be `1.244 0.129 2.376 0.085 NA` for the first day and `17.239 0.530 1.246 0.209 NA` for the second (with more decimals).

Step 20 · So what did the experiment show?

Not needed for the grade, but this is why you did all of this:

	Tapidor	Titlis	p (variety)	p (fertilizer)
Day 1 (14 April)	48.2	50.1	0.29	0.88
Day 2 (15 April)	48.2	**53.2**	**0.002**	0.60

- On day 2, **Titlis had clearly greener leaves than Tapidor** (mean SPAD 53.2 vs. 48.2, p = 0.002).
- **Fertilizer had no detectable effect** on either day (means between 48.6 and 51.3, all p > 0.6). Possible reasons: a small trial (3 replicates), an early stage in the season, or the fact that SPAD readings level off at high nitrogen supply.
- On day 1 nothing was significant. The residual noise on day 1 was twice as large as on day 2 (Sum Sq 133.7 vs. 65.8). The suspicious value of plot 8 is the prime suspect.

The general lesson: **plot your data first** (Steps 12 to 16 already revealed plot 8), and only then trust the p-values.

PART 5 · CHECKLIST

Before you submit

5.1 Where each answer goes

Placeholder in Ex2_R_code.R	Question	Answer type
`#Add answer Q 1 here`	Q1	comments
`#Add answer Q 2 here`	Q2	code + comments
`#Add answer Q 3 here`	Q3	code
(directly below `write_rds(… "dat.long.rds")`)	fix: save dat.longFull.rds	code (1 line)
`#Add answer Q 4 here`	Q4	comments
`#Add the code for Q 5 here` (the **first** one)	Q5	code + `plot(p)`
`#Add the code for Q 5 here` (the **second** one, after `head(dat.wide)`)	**Q6**	comments
`#Type your answer here.`	Q7	comments
`#Type your code here`	Q8	code

5.2 Final checks

- [ ] Every text answer starts with `#` on every line.
- [ ] Q3 sits directly under its placeholder (before the factor step).
- [ ] The extra `write_rds(… "dat.longFull.rds")` line is in place (or the file exists in `out`).
- [ ] Q5 does **not** overwrite `dat.long`; the plot is called `p` and is shown with `plot(p)`.
- [ ] **Restart R and run the whole script from top to bottom.** It must finish without any `Error`.
- [ ] The ANOVA output still shows Df **1, 2, 2, 2, 10**, and your F values match the check in Q8.
- [ ] The `out` folder contains `dat.long.rds`, `dat.longFull.rds`, `rastergraph.png` and `rastergraph.svg`.
- [ ] The script is saved (Ctrl + S) and submitted the way your course describes (the exercise 2 repository on JupyterHub).

APPENDIX A · SPOILERS!

Model answers

TRAP **Try first, then compare**

You learn far more by struggling with a question for ten minutes than by reading the answer. Use this appendix to **check** your work, and write the text answers **in your own words**.

A.1 Q1: comment lines

```
#Add answer Q 1 here
# Q1: measurement.csv has 3 comment lines (the lines starting with "#" above the header).
# They contain metadata, i.e. information ABOUT the data: who prepared the file
# (Andreas Hund, based on original data from Emmanuel Frossard), its purpose (demonstrating
# data handling in R) and its content (ETH plant nutrition excursion, 14 and 15 April 2014).
# Comment lines can be used to document a data file directly inside it: author, source,
# date, units, methods, purpose or notes. read_csv(comment = "#") skips them, so they do
# not disturb the table.
```

A.2 Q2: unique keys

```
#Add answer Q 2 here
design |>
  count(plot.id) |>
  filter(n > 1)
# -> A tibble: 0 x 2. No plot.id occurs more than once: plot.id is a unique key of design.

trait |>
  count(trait.id) |>
  filter(n > 1)
# -> A tibble: 0 x 2. No trait.id occurs more than once: trait.id is a unique key of trait.
```

A.3 Q3: join the method table (plus the missing-file fix)

```
#Add answer Q 3 here
dat.long <- dat.long |>
  left_join(method, by="method.id")
```

and, further down, directly below `write_rds(dat.long, file=paste0(outputPath, "dat.long.rds"))`:

```
write_rds(dat.long, file=paste0(outputPath, "dat.longFull.rds"))
```

A.4 Q4: PNG vs. SVG

```
#Add answer Q 4 here
# The PNG is a raster (bitmap) graphic: a fixed grid of pixels (e.g. 2100 x 2100 px
# for 7 x 7 inches at 300 dpi). When zooming in, the text and edges become pixelated
# or blurry, and single elements (labels, tiles) cannot be edited separately.
# The SVG is a vector graphic: it stores the shapes and texts as instructions/coordinates
# (plain XML text you can open in a text editor). It stays sharp at any zoom level, the
# text remains real, searchable and editable text, and every element can be changed
# in programs like Inkscape or Illustrator. Here the SVG is also smaller (~21 kB vs ~157 kB).
# For further processing or scaling I would prefer the SVG.
```

A.5 Q5: value vs. fertilizer

```
#Add the code for Q 5 here
p <- dat.long |>
  mutate(fertilizer = as.numeric(as.character(fertilizer))) |>  # factor -> real numbers
  ggplot(aes(x = fertilizer, y = value)) +
  geom_point(aes(colour = factor(block))) +
  facet_grid(date ~ variety) +
  theme(axis.text.x = element_text(angle = 90))
plot(p)
```

Variations that also work: `scale_color_discrete()` (the default anyway), or putting `colour = factor(block)` into the main `aes()`. Leaving fertilizer as a factor gives an almost identical plot with evenly spaced 0/90/180. That plot is not wrong as such, but it does not match the target.

A.6 Q6: names_from

```
#Q6:
# pivot_wider took the values of the three columns trait.label, method.label and date
# and pasted them together, separated by "_", to form the NAMES of new columns:
# "Fl0Chl_SPAD001_2014-04-13" and "Fl0Chl_SPAD001_2014-04-14" (one per combination,
# here one per measuring date). The SPAD numbers from the column "value" were moved
# into these new columns. The columns trait.label, method.label, date (and value) no
# longer exist; their information is now stored in the column headers. Because the two
# measurements of each plot now sit side by side in one row, the table shrank from
# 36 rows (one per measurement) to 18 rows (one per plot).
```

A.7 Q7: split()

```
#Type your answer here.
# Q7: split() returns a LIST (not a tibble) with 2 elements, one for each unique date.
# The elements are named after the dates, "2014-04-13" and "2014-04-14". Each element
# is a tibble with 18 rows (the 18 plots measured on that day) and the same 18 columns
# as dat.long. Together the two tibbles contain all 36 rows; the data are only divided
# into groups, nothing is lost. Such a list can then be processed element by element
# with map() (one ANOVA per date).
```

A.8 Q8: F values

```
#Type your code here
F.val <- aov.tab |> map(function(x){x[[1]]$`F value`})
F.val
```

WHAT YOU SHOULD SEE

```
$`2014-04-13`
[1] 1.24400422 0.12872736 2.37648907 0.08533331         NA

$`2014-04-14`
[1] 17.2390982  0.5295666  1.2464561  0.2089240         NA
```

Optional, and nicer: keep the term names, using the row names of each ANOVA table (`trimws` removes the padding spaces R adds to them):

```
F.named <- aov.tab |> map(\(x) setNames(x[[1]]$`F value`, trimws(rownames(x[[1]]))))
F.named
```

APPENDIX B · TROUBLESHOOTING

Error messages and how to fix them

If you see…	it means…	fix
`there is no package called 'tidyverse'` (or `'svglite'`)	the package is not installed	`install.packages("tidyverse")` (or `"svglite"`) once, in the console
`could not find function "read_csv"` (or `"ggplot"`, `"week"`, …)	the tidyverse is not loaded in this session	run `library(tidyverse)`
`could not find function "Mean"`	typo: R is case-sensitive	write `mean`
`'../data/measurement.csv' does not exist in current working directory`	R is not standing in `Scripts`	set the working directory (Part 3), check with `getwd()`
`object 'dat.long' not found`	the line that creates it was not run, there is a typo, or R was restarted	run the script from the top
`cannot open file '../out/dat.longFull.rds': No such file or directory`	the file is missing from the zip	add the `write_rds(… "dat.longFull.rds")` line (Step 11)
`Can't join x$method.id with y$method.id due to incompatible types`	Q3 was placed after the factor step	move it under its placeholder, restart, rerun
`Can't subset columns that don't exist. Column method.label doesn't exist.`	the method table was not joined (Q3 missing or wrong)	do Q3, restart, rerun
`mapping must be created by aes()` / `Did you use %>% or |> instead of +?`	`|>` used between ggplot layers	use `+` after `ggplot(…)`
`Cannot use + with a single argument` / `Did you accidentally put + on a new line?`	a line starts with `+`	put `+` at the **end** of the previous line
`unexpected symbol`, `unexpected numeric constant`, `unexpected ')'`	a missing comma, bracket or quote	check the line character by character
`unexpected end of input`, or the console shows `+`	a bracket or quote was never closed	press Esc, fix, rerun
curly quotes `“ ”` in your code	copied from a PDF or word processor	retype them as straight quotes `"`
`Saving 7 x 7 in image`	not an error, just information	—
ANOVA shows Df **1** for fertilizer	fertilizer is a number, not a factor	do not overwrite `dat.long` in Q5; restart and rerun

APPENDIX C · REFERENCE

Cheat sheet: every function in this exercise

Function	What it does	Example from the exercise
`rm(list = ls())`	delete all objects from memory	Step 1
`dir()`, `getwd()`, `setwd()`	list files / show / set the working directory	Steps 1, 4
`Sys.setenv(TZ = …)`	set the time zone	`"Etc/GMT-1"`
`paste0()`, `paste(…, sep = )`	glue text together (without / with a separator)	`paste0(inputPath, "entry.csv")`
`library()`, `install.packages()`	load / install a package	`library(tidyverse)`
`read_csv(file, comment = "#")`	read a CSV file into a tibble, skipping comment lines	Steps 4, 6
`read_lines(file, n_max = )`	read a file as plain lines of text	Q1
`head()`, `str()`, `names()`, `dim()`	look at a table	everywhere
`select(a, b)`, `select(-a)`, `select(contains("x"))`	keep / drop columns / keep columns whose name contains "x"	Steps 6, 15, 16
`filter(condition)`	keep rows that meet a condition	`filter(n > 1)`, `filter(DOY == 104)`
`count(col)`	how often each value occurs (column `n`)	`count(entry.id)`
`left_join(y, by = "key")`	add the columns of table y by matching a key	Step 8, Q3
`mutate(new = …)`, `across(all_of(cols), f)`	create or change columns; `across` applies f to several columns at once	Steps 9, 11
`as.factor()`, `as.character()`, `as.numeric()`	change the type of a vector	Step 9, Q5
`grep(pattern, x)`	positions of the elements of x that match a pattern	Step 9
`strptime(x, "%Y.%m.%d")`, `as.POSIXct()`	text → date-time	Step 10
`as.Date()`	date-time → date (in UTC unless `tz` is given!)	Step 10
`week()`, `quarter()`, `yday()`	week, quarter, day of year (lubridate)	Step 10
`write_rds()`, `read_rds()`	save / load an R object as .rds	Steps 11, 15
`ggplot(data, aes(x, y, …))`	start a plot and map columns to aesthetics	Steps 12–14
`geom_raster()`, `geom_text()`, `geom_line()`, `geom_point()`	tiles, text, lines, points	Steps 12–14
`facet_grid(rows ~ cols)`	split a plot into panels	`facet_grid(date ~ variety)`
`theme(axis.text.x = element_text(angle = 90))`	rotate the x-axis labels	Steps 13, 14
`ggsave(p, filename = …)`	save a plot to a file (type from the extension)	Step 12
`pivot_wider(names_from, values_from)`	long → wide	Step 15
`plot()`, `cor()`	base-R quick plot (scatter plot for 2 columns); correlation coefficient	Step 16
`split(df, f)`	cut a table into a list of tables, one per group	Step 17
`map(list, f)`	apply f to every list element; returns a list	Steps 18, 19, Q8
`function(x) …`, `\(x) …`	an anonymous (unnamed) function	Step 18
`aov(y ~ a * b + block, data)`, `summary()`	fit an ANOVA model; show its ANOVA table	Steps 18, 19
`x[[1]]`, `x$name`, `x$`odd name``	take an element of a list, or a column	Step 19, Q8

APPENDIX D · WORDS

Glossary

Term	Meaning
Aesthetic (`aes`)	In ggplot: the link between a column and a visual property (x, y, colour, fill, label).
Argument	An input of a function, e.g. `digits = 2` in `round()`.
Block	A group of similar, neighbouring plots containing every treatment once (here 3 blocks).
Comment	Text after `#`. Ignored by R and by `read_csv(comment = "#")`.
Console	The window where R runs commands and prints results.
Data frame / tibble	A table: rows = observations, columns = variables. A tibble is the tidyverse version.
Degrees of freedom (Df)	Number of independent pieces of information, e.g. number of groups − 1.
DOY	Day of year: 1 = 1 January, 104 = 14 April 2014.
Entry	Here: one variety × fertilizer combination (6 entries).
F value	In an ANOVA: variance explained by a factor ÷ residual variance.
Facet	One panel of a plot that is split by a variable.
Factor / level	A categorical variable; its possible values are its levels. Stored internally as codes 1, 2, 3, ….
Foreign key	A column that refers to the primary key of another table.
Function	A named piece of code that takes inputs and returns a result, e.g. `mean()`.
Geom	In ggplot: the type of mark that is drawn (point, line, tile, text).
Join (left join)	Combining two tables by matching key values; a left join keeps all rows of the left table.
List	A container for several objects of any type, accessed with `[[ ]]`.
Long / wide format	One row per measurement vs. one row per unit with one column per measurement.
NA	"Not available": a missing value.
Object	Anything stored under a name in R, created with `<-`.
Ontology	A standardized vocabulary with fixed IDs, e.g. the Crop Ontology for traits and methods.
p-value	The probability of a result at least this extreme if there were no real effect.
Package	An add-on collection of R functions (e.g. dplyr, ggplot2).
Pipe (`|>`)	"And then": passes the result on the left into the function on the right.
Plot	In field trials: the smallest piece of land receiving one treatment (careful: also the word for a graph!).
POSIXct	R's date-time type: seconds since 1 January 1970.
Primary key	A column whose values are unique and identify each row.
Raster / vector graphic	An image made of pixels (PNG) vs. one made of drawing instructions (SVG).
RCBD	Randomized complete block design: every treatment once per block, randomized within the block.
Script	A text file with R code (`.R`), your saved recipe.
SPAD	A hand-held meter giving a relative index (0–99.9) of leaf chlorophyll content.
Working directory	The folder R is currently "standing in". Relative paths start from here.

Every piece of code and output in this guide was run with R 4.3.3 and tidyverse 2.0.0 on the files from Ex2_folders.zip.